PBL SEM-I · JAVA PROJECT

Household Water-Usage & Billing Monitor

From meter reading capture to automated household billing.

Student: Kundana Reddy

Roll No.: 2620030189

Lecturer: Sreeram Murthy

Made with GAMMA

THE PROBLEM

Manual billing creates avoidable errors

Project objective

Households need a simple way to store customer details and meter readings; the solution should show usage, calculate a bill, and preserve billing history. Reduce manual work and provide an easy-to-use water billing workflow.

Remove manual drift

Manual calculations are slow, inconsistent, and difficult to audit.



Made with GAMMA

USER WORKFLOW

Users enter one reading and receive a complete bill

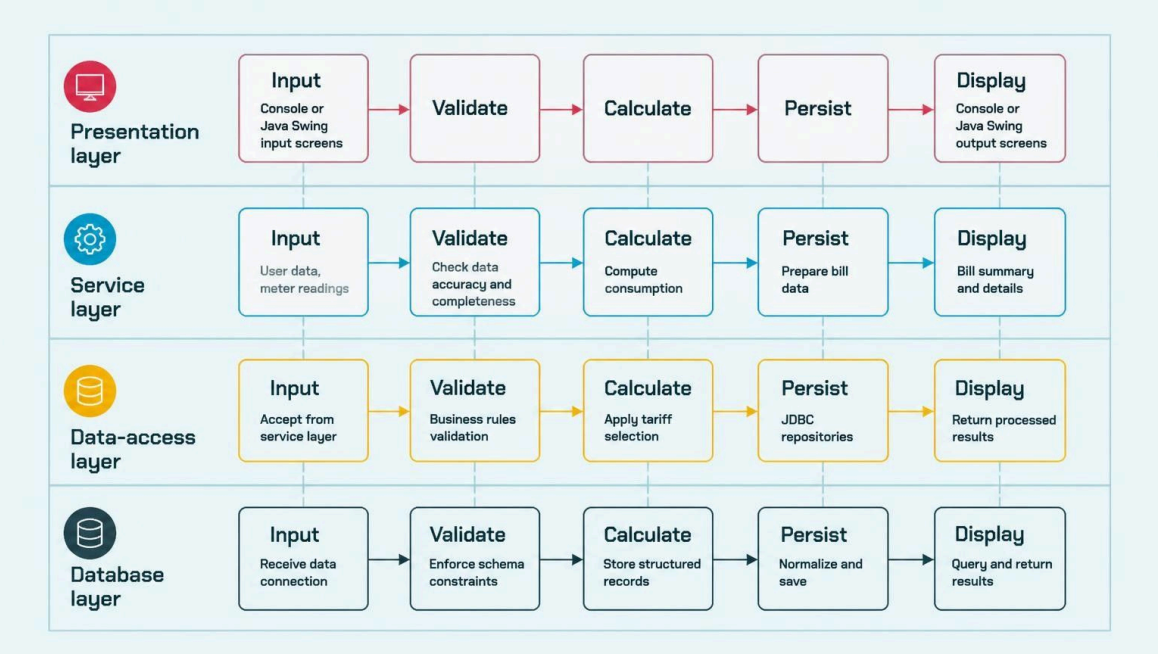
1. Register household details and account identifier.
2. Enter previous and current meter readings.
3. Validate that the reading is numeric and non-decreasing.
4. Compute consumption as current reading minus previous reading.
5. Apply configurable tariff slabs, fixed charges, and optional taxes.
6. Display, save, and retrieve the bill for later review.

Made with GAMMA

APPLICATION ARCHITECTURE

A layered design keeps the application easy to maintain

Each layer owns one responsibility, while the workflow stays visible from input to output.



Made with GAMMA

PERSISTENT DATA MODEL

The database stores history instead of overwriting it

Foreign keys connect readings and bills to households, so every transaction remains traceable.

TABLE	PURPOSE	KEY RELATIONSHIP
households	Customer master data	household_id
meter_readings	Meter history	household_id → households
tariff_slabs	Configurable pricing	slab_id
bills	Generated invoice record	household_id → households

Core relational schema · history is appended as new readings and bills are generated.

Made with GAMMA

ILLUSTRATIVE SAMPLE

266.00

BEFORE TAX

18 units × sample rate 12.00 + sample fixed charge 50.00.

Tariff values are configurable sample data, not official utility rates.

BILLING LOGIC

The bill is driven by a transparent calculation pipeline

$$units = currentReading - previousReading$$

Measure

Compute consumption from two readings.

Price

Charge each unit block at its configured rate.

Finalize

subtotal + fixed charge + tax – approved adjustment.

Made with GAMMA

DOMAIN MODEL

Java classes keep business rules separate from input screens

Class	Responsibility
Household	Identity and contact fields.
MeterReading	Date, value, and validation state.
TariffSlab	Unit range and price rules.
Bill	Usage, subtotal, tax, total, and payment status.
BillingService	Calculates consumption and produces a Bill object.

CORE CALCULATION · JAVA

```
BigDecimal calculateUnits(BigDecimal previous, BigDecimal current) {
  if (previous == null || current == null || current.compareTo(previous) < 0)
    throw new IllegalArgumentException("Current reading must be non-decreasing");
  return current.subtract(previous);
}
```

The service validates before it calculates. BigDecimal keeps currency and usage arithmetic explicit.

Made with GAMMA

DATA ACCESS

JDBC connects the service layer to persistent data

PreparedStatement keeps SQL parameterized and safe; try-with-resources closes every resource.

INSERT READING · JAVA + JDBC

```
try (Connection con = DriverManager.getConnection(URL, USER, PASS);
     PreparedStatement ps = con.prepareStatement(
        "INSERT INTO meter_readings "
      + "(household_id, reading_date, meter_value) VALUES (?, ?, ?)")) {
    ps.setString(1, householdId);
    ps.setDate(2, Date.valueOf(readingDate));
    ps.setBigDecimal(3, meterValue);
    ps.executeUpdate();
}
```

Made with GAMMA

ACCURACY CONTROLS

Validation and exceptions protect billing accuracy

Input check	System response
Blank household ID; invalid date	Reject input; require account identity and protect billing period
Negative reading; current < previous	Reject input; protect meter data and prevent reverse usage

Currency: Use BigDecimal rather than floating-point arithmetic.

Validation: `if (id.isBlank() || date == null || prev.signum() < 0 || curr.signum() < 0 || curr.compareTo(prev) < 0) throw new IllegalArgumentException();`

ERROR HANDLING

Handle SQLException, duplicate account IDs, missing tariff slabs, and invalid numeric input.

```
catch (SQLException | NumberFormatException e) { log(e); showUserMessage("Unable to create bill"); }
```

Return user-friendly messages while logging technical details for debugging.



Made with GAMMA

Thank You

Household Water-Usage & Billing Monitor

PBL SEM-I · Java Project · KLH University

Student: Kundana Reddy

Roll No.: 2620030189

Lecturer: Sreeram Murthy

Made with GAMMA